Hands-on Assignment: Sequential Multi-Agent System Using Microsoft Agent Framework

Technology: Python, PyCharm, Microsoft Agent Framework (MAF), OpenAI GPT-4o-mini

Objective

Build an **E-commerce AI Shopping Assistant** using Microsoft Agent Framework and `SequentialBuilder`.

The application must use three specialized AI agents that work sequentially to search products, calculate prices, and recommend suitable products based on customer requirements.

Business Scenario

An e-commerce company wants to automate product recommendations using multiple AI agents.

Customers should be able to ask questions such as:

"Recommend 2 laptops within a total budget of ₹1,20,000."

The system must search the available products, verify stock, calculate the total cost, and recommend the most suitable product.

Task 1 – Project Setup

Create the following Python project:

```
Ecommerce_Multi_Agent/
├── .env
├── requirements.txt
├── products.json
├── tools.py
```

```
├── agents.py
└── sequential.py
```

Required Python packages:

```
agent-framework-openai
agent-framework-orchestrations
python-dotenv
```

Install the dependencies and configure your OpenAI API key in `.env`.

Use GPT-4o-mini as the language model.

Task 2 — Understand the Dataset

Use the supplied `products.json` file containing 32 fictional products across multiple categories, including laptops, smartphones, tablets, monitors, headphones, printers and accessories.

Each product contains:

- Product ID
- Product name
- Category
- Price in INR
- Available stock quantity
- Customer rating

The application must retrieve product information from this JSON file instead of hardcoding product information in the agent instructions.

Task 3 — Create Python Tools

Create two Python functions inside `tools.py`.

Tool 1: `search_products()`

Requirements:

- Accept a product name or category as input.
- Search the JSON dataset.
- Return matching products.
- Include product ID, price, stock and rating.

- Return an appropriate message when no products match.

Tool 2: `calculate_order_total()`

Requirements:

- Accept product ID and requested quantity.
- Retrieve the product price from JSON.
- Calculate the total cost.
- Verify that sufficient stock is available.
- Reject invalid quantities.
- Return an appropriate message if the product does not exist.

Test both functions independently before attaching them to agents.

Task 4 — Create Three AI Agents

Use Microsoft Agent Framework to create the following agents.

Agent 1 — Inventory Agent

Responsibilities:

- Understand the customer's product requirements.
- Use the product search tool.
- Identify matching products.
- Provide product details and stock availability.
- Pass the relevant information to the next agent.

Agent 2 — Pricing Agent

Responsibilities:

- Receive the results from the Inventory Agent.
- Use the pricing tool to calculate costs.
- Validate requested quantities against available stock.
- Compare calculated totals with the customer's budget.
- Identify affordable products.

Agent 3 — Recommendation Agent

Responsibilities:

- Review the inventory and pricing results.
- Recommend the highest-rated product that meets the customer's budget and quantity requirements.
- If ratings are equal, prefer the lower-cost option.
- Provide a short explanation for the recommendation.
- Do not recommend unavailable products.

Task 5 — Implement Sequential Orchestration

Use Microsoft Agent Framework's `SequentialBuilder` to create the following workflow:

```
Customer Request
       |
       v
 Inventory Agent
       |
       v
  Pricing Agent
       |
       v
Recommendation Agent
       |
       v
 Final Recommendation
```

Requirements:

1. Execute the three agents in the specified order.
2. Pass relevant conversation context between agents.
3. Allow each agent to use its assigned tools.
4. Execute the workflow asynchronously.
5. Display the intermediate results of all three agents.
6. Display the final recommendation.

Important: Implement the orchestration using `SequentialBuilder`. Do not manually call three agents one after another using separate `run()` statements.

Task 6 — Test the Multi-Agent System

Run the application with the following customer requests.

Test	Customer Request
1	Recommend 2 laptops within ₹1,20,000 total.
2	Find 3 smartphones within ₹1,00,000 total.
3	Recommend 2 monitors within ₹50,000 total.
4	Find 5 headphones under ₹20,000 total.
5	Recommend 2 tablets under ₹60,000 total.
6	Find 10 printers within ₹1,00,000 total.
7	Find 100 laptops and verify stock availability.
8	Recommend a product category that does not exist in the dataset.

Verify that the agents handle insufficient stock, excessive prices, and unavailable products appropriately.

Expected Output

For every customer request, the terminal should clearly display:

Inventory Agent

- Matching products
- Product IDs
- Available stock

Pricing Agent

- Selected product IDs
- Quantity requested
- Calculated total cost
- Budget validation

Recommendation Agent

- Recommended product
- Price and quantity
- Customer rating
- Reason for recommendation